

Implementation of a Comprehensive Testing Strategy for the alf.io Open-Source Event Ticketing Platform

Christian Raúl Mestas Zagarra
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
cmestasz@unsa.edu.pe

Luis Gustavo Sequeiros Condori
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
lsequeiros@unsa.edu.pe

Mariel Alisson Jara Mamani
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
mjarama@unsa.edu.pe

Rodrigo Alexander Fernández
Huarca
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
rfernandezh@unsa.edu.pe

Álvaro Raúl Quispe Condori
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
aquispecondo@unsa.edu.pe

Mathías Alonso Barrios Medina
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
mbarriosmed@unsa.edu.pe

Abstract—This paper presents the implementation of a comprehensive testing strategy for alf.io, an open-source event ticketing platform. The project involved designing and executing tests across four levels: unit, integration, system, and acceptance testing. The testing covered the entire technology stack including a Java/Spring Boot backend, Angular public frontend, and Lit-based administration interface. Automated testing through GitHub Actions CI/CD pipelines achieved 100% pass rates across unit tests (2,290 cases), integration tests (442 cases), and end-to-end tests (88 cases), with an aggregated code coverage of 85.76%. Performance testing with K6 validated system behavior under load, fuzzing with Jazzer confirmed input handling robustness, and static analysis with SonarQube identified code quality improvements. The results demonstrate the effectiveness of multi-layered testing approaches for ensuring quality in complex web applications.

Index Terms—Software Testing, Test Automation, CI/CD, Open Source, Event Management, Quality Assurance

I. INTRODUCTION

Software testing has become an indispensable practice in modern software engineering, ensuring that systems meet quality standards and function correctly under various conditions [1]. The complexity of contemporary web applications, which integrate multiple technologies, databases, and external services, demands comprehensive testing strategies that go beyond simple unit tests [2]. As systems grow in complexity, the need for systematic testing approaches that cover all levels becomes increasingly critical [3].

The event management and ticketing domain presents unique challenges for software testing. Systems like alf.io must handle concurrent reservations, process payments through

multiple gateways, manage digital ticket generation, and provide real-time check-in capabilities [4]. These requirements create complex interactions between components that can only be validated through rigorous testing at multiple levels. The financial nature of ticketing transactions adds another layer of complexity, as errors can result in revenue loss and damaged reputation.

This paper presents the implementation of a comprehensive testing strategy for alf.io, an open-source event ticketing platform. The project involved designing and executing tests across four distinct levels: unit, integration, system, and acceptance testing [5]. Each level addressed specific quality attributes and used specialized tools and methodologies to validate different aspects of the system's behavior.

The testing implementation covered the entire technology stack of alf.io, including the Java/Spring Boot backend [6], Angular public frontend, and Lit-based administration interface. The strategy incorporated automated testing through GitHub Actions CI/CD pipelines [7], performance testing with K6 [8], fuzzing with Jazzer [9], and static analysis with SonarQube [10]. This multi-layered approach ensured comprehensive coverage of both functional and non-functional requirements.

The results demonstrate the effectiveness of the implemented testing strategy, achieving 100% pass rates across unit, integration, and end-to-end tests, with an aggregated code coverage of 85.76% [11]. The project also identified and documented defects, validated API contracts, and established reproducible testing processes through automated pipelines [12]. This work provides a practical reference for implement-

ing comprehensive testing strategies in similar open-source web applications [13].

II. BACKGROUND

A. State of the Art in Software Testing

Software testing has evolved significantly over the past decades, transitioning from manual verification to sophisticated automated frameworks and methodologies [3]. The ISO/IEC/IEEE 29119 standard provides a comprehensive framework for software testing, defining concepts, processes, documentation, and techniques that guide testing activities across all levels of software development. This standard emphasizes the importance of systematic test planning, execution, and reporting to ensure software quality.

The testing pyramid model, first proposed by Mike Cohn, remains a foundational concept in test strategy design [5]. This model suggests that teams should invest most heavily in unit tests, followed by integration tests, with fewer end-to-end tests at the top. Modern adaptations of this model consider additional layers such as contract testing and performance testing [14].

Test automation has become a cornerstone of modern software development practices, particularly in the context of continuous integration and continuous deployment pipelines [12]. Research has demonstrated that automated testing significantly reduces time-to-market while improving code quality [15]. Studies have shown that defects found in production cost significantly more to fix than those identified during development.

Modern testing strategies encompass multiple specialized approaches beyond traditional functional testing. Performance testing tools like K6 enable teams to validate system behavior under various load conditions [8]. Security testing through fuzzing tools such as Jazzer can identify vulnerabilities that traditional testing might miss [9]. Static analysis tools like SonarQube provide continuous code quality monitoring, detecting bugs, vulnerabilities, and code smells before they reach production [10].

B. Event Management Systems and Open Source Software

Event management systems represent a specialized category of web applications that must handle complex business logic, financial transactions, and real-time operations. The open-source model has proven particularly successful in this domain, as it allows organizations to customize and extend the software to meet their specific needs while avoiding vendor lock-in [16].

The alf.io project, created in 2014 by Celestino Bellone in Ticino, Switzerland, exemplifies the evolution of open-source event management software [4]. What began as a simple Spring Boot application has grown into a mature ecosystem with 1,596 GitHub stars, 141,000+ Docker Hub pulls, and 100 releases. The project's name derives from the Italian given name Alfio, reflecting its origins in the Italian-speaking region of Switzerland.

The technology stack of alf.io has evolved significantly over its 12-year history. The backend transitioned from Spring Boot

1.x to 3.x, with Java evolving from version 8 to 17 [6]. The frontend underwent a major transformation from AngularJS to Lit web components, reflecting the broader industry trend toward lightweight, standards-based frameworks. The database layer consolidated from supporting multiple databases to PostgreSQL-only, leveraging advanced features like Row-Level Security for multi-tenancy.

The project's architecture follows a modern microservices-inspired design while maintaining a monolithic deployment model. The backend implements a layered architecture with controllers, managers, repositories, and models. The frontend consists of two separate single-page applications: a public-facing Angular application for ticket purchases and a Lit-based administration interface for event organizers.

Payment gateway integration represents a critical aspect of event management systems. Alf.io supports multiple payment providers including Stripe, PayPal, Mollie, and bank transfers. The system's extension mechanism allows for custom integrations and business logic, making it adaptable to various organizational requirements.

III. STUDY CASE

A. Project History and Evolution

Alf.io was created on July 26, 2014, by Celestino Bellone, a software developer based in Rovio, Switzerland [4]. The project emerged from the need for a self-hosted, open-source ticketing solution that could be customized for various event types. Bellone, who later co-founded Swicket, a premium hosted version of alf.io, has contributed 2,531 commits to the project, representing approximately 64% of all contributions.

The project's co-maintainer, Sylvain Jermini, joined in 2010 and has contributed 1,295 commits, accounting for 33% of total contributions. Jermini, based in Switzerland, has been instrumental in the frontend architecture evolution, particularly the migration from AngularJS to Lit web components. Together, Bellone and Jermini have maintained continuous development over 12 years, releasing 100 versions and establishing alf.io as a mature, production-ready solution [13].

The project's evolution can be traced through major version milestones. Table I summarizes the key versions and their features.

TABLE I
KEY VERSIONS AND MILESTONES OF ALF.IO.

Version	Date	Key Features
1.0	2014	Initial release
1.14	Apr 2018	Extension system with scripting hooks
2.0-M0	Nov 2018	Spring Boot 2, MySQL/HSQldb removed
2.0-M4	Apr 2022	Organizations, multi-tenancy, Stripe/Mollie/PayPal
2.0-M5	Sep 2024	Spring Boot 3.2, Lit admin frontend, Cloudflare Turnstile
2.0-M5-2606	Jun 2026	Security fixes, custom offline payments, pnpm frontend

B. Technology Stack

The current technology stack of alf.io reflects modern web application development practices [6]. Table II summarizes the main components.

TABLE II
TECHNOLOGY STACK OF ALF.IO.

Component	Technology
Backend	Java 17, Spring Boot 3.5.x, Jetty
Build System	Gradle with test automation integration
Database	PostgreSQL 10/15/16 with Flyway migrations
Public Frontend	Angular 17
Admin Frontend	Lit 3 with Shoelace, Vite
Unit Testing	JUnit 5
Integration Testing	Testcontainers [17]

C. System Architecture

The system architecture follows a layered design pattern with clear separation of concerns. The backend implements a standard Spring Boot architecture with controllers handling HTTP requests, managers implementing business logic, repositories managing database interactions, and models representing domain entities [6].

The database schema implements Row-Level Security in PostgreSQL to ensure data isolation between organizations in multi-tenancy deployments. This approach provides database-level security without requiring separate database instances for each tenant. The schema is version-controlled through Flyway migrations, ensuring consistent database state across development, testing, and production environments.

External service integrations include payment gateways, email services, and digital wallet providers. The system supports both online and offline payment methods, with built-in mechanisms for handling payment confirmation, expiration, and reconciliation.

D. Fork and Academic Context

The repository under study, catarinas-ps-2026/alf.io, is a fork of the original alfio-event/alf.io project, created for academic purposes. The fork was established on May 24, 2026, as part of a software testing course at the Universidad Nacional de San Agustín de Arequipa. The project team consists of six students led by a test lead, with Professor Robert Edison Arisaca Mamani providing academic supervision.

The academic context required a systematic approach to testing that went beyond typical open-source contribution practices [16]. The team implemented comprehensive test plans following ISO/IEC/IEEE 29119 standards, with formal documentation of test strategies, execution results, and defect tracking [3].

IV. METHODS

The testing strategy followed a structured approach with seven major activities, each addressing a specific aspect of the

quality assurance process. This section describes each activity in detail.

A. Planning and Methodology

Test planning followed the ISO/IEC/IEEE 29119 standard, with formal test plans documenting scope, methodology, entry and exit criteria, and risk mitigation strategies [3]. The project adopted Scrum as the development framework, with two-week sprints and clearly defined deliverables for each testing level [5]. Communication was maintained through WhatsApp for daily coordination, Google Meet for sprint ceremonies, and GitHub Projects for task tracking.

Each testing level had its own test plan and execution report, providing traceability from requirements to test results. The team conducted sprint planning, review, and retrospective meetings to ensure alignment and continuous improvement.

Risk management was integrated into the testing process, with a formal risk register identifying potential issues and their mitigation strategies. Table III summarizes the key risks identified.

TABLE III
KEY RISKS IDENTIFIED AND THEIR MITIGATION STRATEGIES.

Risk	Prob.	Impact	Severity	Mitigation
Limited Spring Boot experience	4	4	16	Peer code reviews
Tight delivery deadline	5	4	20	Prioritize critical modules
Integration conflicts	4	3	12	Strict branching policies
Tool automation difficulties	2	3	6	Internal technical support

B. Unit Testing

Unit testing focused on validating individual components in isolation, using mocks and test doubles to eliminate external dependencies [2]. The backend tests used JUnit 5 with Mockito for dependency mocking, while frontend tests used Vitest with HttpTestingController for Angular and jsdom for Lit components [6].

The test suite was organized by application layer: managers, controllers, repositories, utilities, and frontend components. Backend unit tests covered business logic managers, REST API controllers, data repositories, and utility classes. Frontend unit tests validated component behavior, service communication, route guards, and form validations.

The test execution was automated through GitHub Actions [7], with two workflows: test-pr.yml for pull requests and test-push.yml for main branch pushes. Coverage was measured using JaCoCo for backend and Vitest's coverage-v8 for frontend [11].

C. Integration Testing

Integration testing validated the interaction between multiple components using real dependencies [17]. The strategy adopted a Big Bang incremental approach, first integrating

infrastructure components, then business logic flows, and finally API contract validation.

Backend integration tests used Testcontainers to manage ephemeral PostgreSQL instances, providing real database interactions without external dependencies. The tests validated complete flows from controllers through managers to repositories. Stripe Mock container was used for payment gateway testing, while PayPal sandbox accounts enabled real payment flow validation.

API contract validation was performed using SpringDoc OpenAPI to generate descriptors automatically from controllers. End-to-end testing with Playwright validated the administration interface against the running backend [18].

D. Functional Testing

Functional testing employed black-box testing techniques to validate system behavior against requirements without examining internal code structure [2]. The techniques applied included equivalence partitioning, boundary value analysis, use case testing, and decision tables [3].

The project was tasked with selecting three functional modules for focused testing. The selected modules were: Manager, Controller/API, and Reservation Flows. These modules represent the core business logic, API layer, and critical user workflows of the system respectively.

The functional test suite covered manual test cases across the three selected modules, with additional coverage of supporting modules. Each test case followed a structured format with preconditions, input data, execution steps, and expected results.

E. System Testing

System testing focused on non-functional requirements, specifically security and performance [1]. The security testing strategy employed two complementary approaches: static application security testing with SonarQube [10] and fuzzing with Jazzer [9]. Performance testing used K6 to validate system behavior under various load conditions [8].

Table IV summarizes the tools used for system testing.

TABLE IV
SYSTEM TESTING TOOLS AND THEIR TARGETS.

Tool	Category	Target	Metric
SonarQube	SAST - Security	Static code analysis	Bugs, vulnerabilities, code smells
Jazzer	Fuzzing - Security	35 utility classes	Crashes, exceptions
K6	Performance	Reservation flow	Response time, throughput, error rate

F. Acceptance Testing

Acceptance testing validated complete user workflows from the end-user perspective. The acceptance test suite covered the admin panel interface, including authentication, event management, reservation processing, and check-in operations.

These tests were executed against a running instance of alf.io deployed in a Kubernetes cluster.

The acceptance tests used Playwright for browser automation, testing against both Chromium and Firefox browsers [18]. The tests validated that the system met the specified acceptance criteria and functioned correctly in a production-like environment.

G. CI/CD Automation

The CI/CD automation activity established automated pipelines for continuous testing and deployment [7]. The pipeline included code formatting, style checking, unit testing, integration testing, end-to-end testing, performance testing, fuzzing, and static analysis [12].

Figure 1 shows the CI/CD pipeline workflow with parallel job execution.

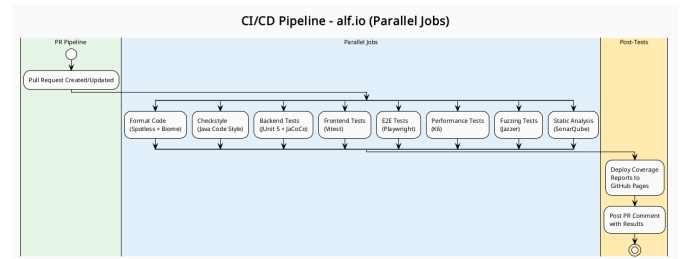


Fig. 1. CI/CD pipeline workflow for alf.io showing parallel job execution.

The pipeline executed on every pull request and push to the main branch, ensuring that all changes were validated before deployment [15]. Coverage reports were automatically generated and published to GitHub Pages for stakeholder review.

V. RESULTS

A. Components Tested

The testing strategy covered all major components of the alf.io system across three applications and supporting infrastructure [13]. Table V summarizes the components and their test counts.

TABLE V
COMPONENTS TESTED AND THEIR RESULTS.

Component	Technology	Test Count	Pass Rate
Backend	Java 17, Spring Boot 3.5.x	2,290	100%
Public Frontend	Angular 17	487	100%
Admin Frontend	Lit 3	167	100%
Integration Backend	JUnit + Testcontainers	354	100%
E2E Tests	Playwright	88	100%
Functional Tests	Manual	120+	97.5%

B. Unit Test Results

The unit test suite executed 2,290 test cases across all three applications, achieving a 100% pass rate. The backend

TABLE VI
FUNCTIONAL TEST RESULTS BY MODULE.

Module	Test Cases	Pass Rate	Defects Found
Manager	15	93%	1
Controller/API	20	95%	1
Reservation Flows	25	96%	1
Ticket Editing	11	82%	3
Event Configuration	7	57%	3
Other Modules	42	100%	0

E. Defects Found and Fixed

The testing process identified and documented defects across all testing levels [3]. Table VII lists the defects discovered during functional testing and the commits that fixed them.

TABLE VII
DEFECTS DISCOVERED DURING FUNCTIONAL TESTING AND THEIR FIXES.

Test ID	Module	Defect	Commit
CPF-01-003	Ticket Editing	Server error 500 with 254-char names	7e524f8
CPF-01-006	Ticket Editing	System accepts numbers in name field	99f17a7
CPF-01-011	Ticket Editing	Email length validation not enforced	dc1341a
CPF-03-006	Reservation State	Button visible with incomplete data	7e524f8
CPF-09-006	Organization Config	SQL error exposed with 256-char names	99f17a7
CPF-10-001	Event Config	Date picker scroll bug	dc1341a
CPF-10-006	Event Config	Silent save disable without feedback	7e524f8
CPF-10-007	Event Config	Duplicate hidden codes allowed	99f17a7
CPF-14-004	Localization	Timezone warning not displayed	dc1341a

F. System Test Results

Figure 8 shows the K6 performance test results [8]. Performance testing executed 54,176 HTTP requests against the reservation flow, achieving a throughput of 190.43 requests per second. The median response time was 745 milliseconds, meeting the less than 2 second target. However, the error rate of 8.43% exceeded the less than 1% target under extreme load conditions.

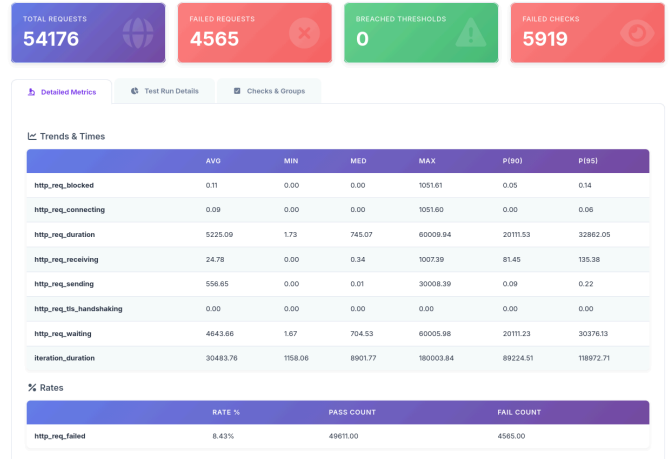


Fig. 8. K6 performance test results.

Figure 9 shows the Jazzer fuzzing test results [9]. Fuzzing testing executed 35 target classes covering JSON deserialization, input validation, cryptographic utilities, template rendering, business logic, and extension processing [19]. The fuzzing campaign found zero critical crashes.

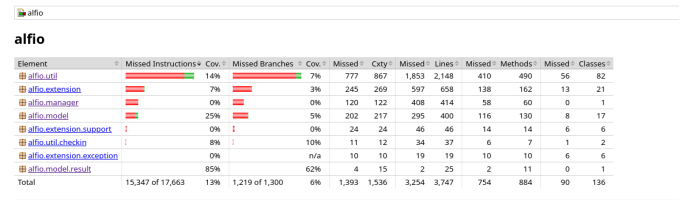


Fig. 9. Jazzer fuzzing test results.

Figure 10 shows the SonarQube static analysis results [10]. Static analysis identified 16 bugs, 10 vulnerabilities, and 1,412 code smells across the codebase. The Quality Gate passed with a technical debt ratio of 0.5% [20].

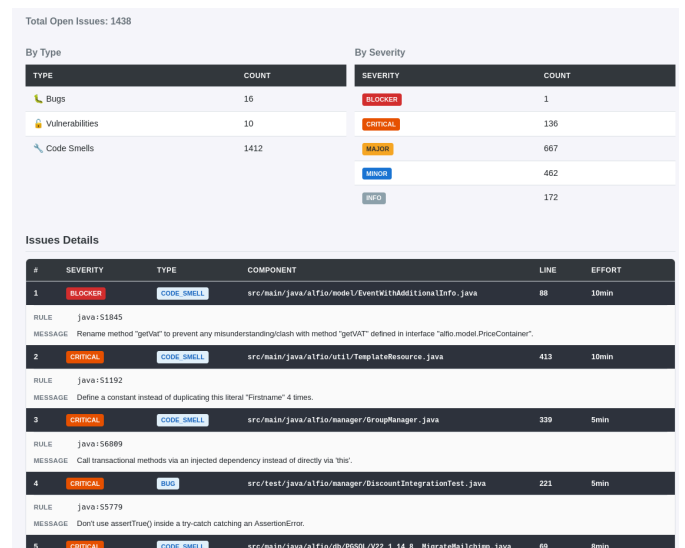


Fig. 10. SonarQube static analysis results.

VI. CONCLUSION

The comprehensive testing strategy implemented for alf.io demonstrated the effectiveness of a multi-layered approach to software quality assurance. The project achieved 100% pass rates across unit tests, integration tests, and end-to-end tests, while maintaining an aggregated code coverage of 85.76%. These results validated the testing methodology and confirmed the system's functional correctness.

The unit testing phase established a solid foundation by validating individual components in isolation. The combination of JUnit 5 with Mockito for backend and Vitest for frontend provided comprehensive coverage of business logic, API controllers, and user interface components. The automated execution through GitHub Actions ensured consistent test results and rapid feedback for developers.

Integration testing confirmed that system components interacted correctly with real dependencies. The use of Testcontainers for ephemeral PostgreSQL instances, Stripe Mock for payment testing, and Playwright for browser automation provided realistic validation of the complete system stack. The API contract validation through OpenAPI and Redoc ensured API stability across versions.

The functional testing phase, conducted through manual execution of 120+ test cases, validated user-facing functionality across 15 modules. This testing identified defects that automated tests could not catch, such as UI behavior inconsistencies and edge cases in user interactions. The systematic application of black-box testing techniques ensured thorough coverage of functional requirements.

System testing addressed non-functional requirements through performance testing with K6, fuzzing with Jazzer, and static analysis with SonarQube. While the performance tests identified a scalability issue under extreme load, the system demonstrated graceful degradation. Fuzzing confirmed robust handling of invalid inputs with zero critical crashes, and SonarQube identified areas for code quality improvement with an acceptable technical debt ratio of 0.5%.

The implementation established reproducible testing processes through automated CI/CD pipelines, comprehensive documentation following ISO/IEC/IEEE 29119 standards, and systematic defect tracking. The project demonstrated that comprehensive testing strategies, when properly planned and executed, can significantly improve software quality while providing valuable feedback for development teams working on complex web applications.

VII. FUTURE WORK

The testing strategy implemented for alf.io provides a foundation for continuous improvement and expansion. Several areas have been identified for future enhancement that would further strengthen the quality assurance process.

Performance testing should be expanded to address the scalability issues identified under extreme load conditions [8]. The current K6 tests revealed an error rate that exceeded the target at 1,600 concurrent users. Future work should investigate root causes in the reservation and confirmation endpoints,

implement rate limiting mechanisms, and optimize database queries to improve throughput.

The acceptance testing phase should be expanded to include more complete end-to-end validation of user workflows [18]. This includes automated acceptance tests that simulate real user journeys from event discovery through ticket purchase, check-in, and post-event reporting. Such tests would provide higher confidence in the system's production readiness.

Security testing could be enhanced by expanding the fuzzing coverage to include additional system components [9] and implementing security-focused penetration testing. The current SonarQube analysis identified vulnerabilities that should be addressed, particularly the BLOCKER issue related to method naming conflicts and the CRITICAL issues involving transactional method calls [10].

Test automation coverage should be expanded to include more functional test scenarios that are currently executed manually [2]. Converting manual test cases to automated scripts would improve regression testing efficiency and enable faster release cycles [12]. Priority should be given to high-impact test cases covering payment processing, ticket generation, and check-in operations.

The test infrastructure could benefit from implementation of visual regression testing to detect unintended UI changes. Tools like Percy or Chromatic could be integrated into the Playwright test suite to capture and compare screenshots across browser versions and screen sizes.

Monitoring and observability should be integrated into the testing process to provide real-time insights into system behavior during test execution. Implementing distributed tracing, metrics collection, and log aggregation would enable better diagnosis of test failures and performance issues.

The documentation generated during this project, following ISO/IEC/IEEE 29119 standards, provides a template for future testing efforts [3]. This documentation should be maintained and updated as the system evolves, ensuring that test plans, execution reports, and defect tracking remain current and useful for ongoing quality assurance activities.

REFERENCES

- [1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.
- [2] G. J. Myers, T. Badgett, and T. M. Thomas, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, 2011.
- [3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29119:2021 - Software Testing*. International Organization for Standardization, 2021.
- [4] C. Bellone, "alf.io - Free and Open Source Event Attendance Management," 2014.
- [5] M. Cohn, *Agile Estimating and Planning*. Upper Saddle River, NJ: Prentice Hall, 2004.
- [6] VMware, "Spring Boot Documentation," 2024.
- [7] GitHub, "GitHub Actions Documentation," 2024.
- [8] I. W. R. Pinastawa and others, "Web Server Performance Evaluation: Comparing Nginx and Apache Using K6 Testing Methods for Load, Spike, Soak, and Performance," in *2024 IEEE International Conference on Innovation, Analytics, Intelligent Technologies and Systems (ICI-CiTIS)*, 2024. doi: 10.1109/icimis63449.2024.10956402.
- [9] S. Dechand and others, "Jazzer: Coverage-Guided Fuzzing for Semantic Vulnerabilities in the Java Ecosystem," in *IEEE Symposium on Security and Privacy (S&P)*, 2026. doi: 10.1109/SP63933.2026.00134.
- [10] D. Marcilio and others, "Are Static Analysis Violations Really Fixed? A Closer Look at Realistic Usage of SonarQube," in *2019 IEEE/ACM*

International Conference on Program Comprehension (ICPC), 2019. doi: 10.1109/icpc.2019.00040.

- [11] P. S. Kochhar and others, "Code Coverage and Test Suite Effectiveness: Empirical Study with Real Bugs in Large Systems," in *2015 IEEE 22nd International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2015. doi: 10.1109/saner.2015.7081877.
- [12] S. Elbaum and others, "Techniques for Improving Regression Testing in Continuous Integration Development Environments," in *Proc. 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2016. doi: 10.1145/2635868.2635910.
- [13] M. Aberdour, "Achieving Quality in Open-Source Software," *IEEE Software*, 2007, doi: 10.1109/ms.2007.2.
- [14] V. Mukhin and others, "The Testing Mechanism for Software and Services Based on Mike Cohn's Testing Pyramid Modification," in *2021 IEEE 11th International Conference on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS)*, 2021. doi: 10.1109/idaacs53288.2021.9660999.
- [15] H. Spieker and others, "Reinforcement Learning for Automatic Test Case Prioritization and Selection in Continuous Integration," in *Proc. 2017 10th International Conference on Software Testing, Verification and Validation (ICST)*, 2017. doi: 10.1145/3092703.3092709.
- [16] I. Stamelos and others, "Code Quality Analysis in Open Source Software Development," *Information Systems Journal*, 2002, doi: 10.1046/j.1365-2575.2002.00117.x.
- [17] S. Shahsavan and others, "Testing the Real World: Integration Tests with Testcontainers in Go," 2026, doi: 10.1007/979-8-8688-2370-1_17.
- [18] G. Aryotejo and others, "A Comparative Analysis of Web Automation Testing Tools: Selenium Webdriver, Playwright, and Webdriverio Using an AHP-TOPSIS Approach," in *IEEE International Conference (ICICoS)*, 2025. doi: 10.1109/icicos68590.2025.11330144.
- [19] A. Takanen, J. D. DeMott, and C. Miller, *Fuzzing for Software Security Testing and Quality Assurance*. Artech House, 2008.
- [20] V. Lenarduzzi and others, "Are SonarQube Rules Inducing Bugs?," 2020, doi: 10.1109/saner48275.2020.9054821.